

多波束测线覆盖宽度建模与最短测线优化设计

摘要

本文针对 2023 年高教社杯全国大学生数学建模竞赛 B 题，研究多波束测深系统的覆盖宽度、相邻条带重叠率与测线布设问题。

针对问题一，本文在与测线方向垂直的二维剖面内建立斜坡多波束覆盖的几何模型。以测线位置 x 的水平距离为变量，给出水深 $D(x) = D_0 - x \tan \alpha$ ，由声束边界与坡面交点推导左右半覆盖宽度 B_L, B_R 及覆盖宽度 $W = B_L + B_R$ ，并以“当前条带宽度”为分母定义相邻条带重叠率 $\eta_i = O_i/W(x_i)$ 。代入开角 120° 、坡度 1.5° 、中心水深 70 m，得到覆盖宽度由 -800 m 处的 315.71 m 单调减小至 800 m 处的 170.27 m，相邻重叠率由 35.70% 降至 -12.36% ，说明浅水侧会出现漏测。模型在 $\alpha = 0$ 时退化为平底公式 $W = 2D \tan(\theta/2)$ ，验证了模型正确性。

针对问题二，本文将二维模型推广到三维坡面。通过坡面法向水平投影与测线方向的夹角 β ，给出船位水深 $D(r, \beta) = D_0 + s \tan \alpha \cos \beta$ 与垂直测线剖面的等效坡度 $\alpha_{\text{eff}} = \arctan(\tan \alpha \sin \beta)$ ，再代入二维覆盖宽度公式。结果表明 $\beta = 90^\circ, 270^\circ$ 时船沿等深线运动、覆盖宽度恒为 416.55 m； $\beta = 0^\circ$ 时随距离增加覆盖宽度由 415.69 m 增至 768.48 m，与深浅方向变化一致。

针对问题三，本文在规则单坡面矩形海域内建立以测线总长度最小为目标、以完全覆盖和 10% ~ 20% 重叠率为约束的优化问题。采用“沿等深线南北向布设 + 贪心递推”求解，每步取重叠率下限 10% 以最大化间距，得到 34 条测线，总长度 125936.00 m (68.00 海里)；东西边界均被覆盖且相邻重叠率全部为 10.00%，并验证少一条测线将出现 25.76 m 缺口，说明在该方向下测线数已不可再减少。

针对问题四，本文利用附件 201×251 水深网格估计东西向局部坡度，采用南北向分带递推布线和网格覆盖仿真评价。比较全域、1.00、0.50、0.25 NM 四种分带方案后，推荐 0.50 NM 分带方案：测线线段 461 段，总长度 426886.00 m，漏测率为 0.00%，重叠率超过 20% 的部分总长 5481.92 m，在完全覆盖前提下兼顾了测线长度、冗余与可执行性。

关键字：多波束测深 覆盖宽度 重叠率 几何建模 测线优化 递推布线

目录

一 问题重述	3
1.1 问题背景	3
1.2 问题内容	3
二 问题分析	3
2.1 总体分析	3
2.2 问题一的分析	4
2.3 问题二的分析	4
2.4 问题三的分析	4
2.5 问题四的分析	4
三 模型假设	4
四 符号说明	5
五 问题一的模型建立与求解	6
5.1 建模思路	6
5.2 水深模型	6
5.3 覆盖宽度模型	6
5.4 相邻条带重叠率模型	7
5.5 求解与结果	7
5.6 模型验证与本问结论	8
六 问题二的模型建立与求解	8
6.1 三维坡面到二维剖面的转化	8
6.2 求解与结果	8
6.3 模型验证与本问结论	9
七 问题三的模型建立与求解	10
7.1 优化模型的建立	10
7.2 贪心递推求解	10
7.3 求解结果	11
7.4 模型验证与本问结论	11

八 问题四的模型建立与求解	12
8.1 数据说明与局部坡度估计	12
8.2 分带递推布线与覆盖评价	12
8.3 候选方案比较与推荐方案	12
8.4 模型验证与本问结论	14
九 敏感性分析	14
9.1 问题一覆盖宽度对坡度与开角的敏感性	14
9.2 问题三最小测线数的约束敏感性	14
9.3 问题四分带粒度的敏感性	15
十 模型评价与推广	15
10.1 模型优点	15
10.2 模型不足	15
10.3 模型推广	16
参考文献	17
附录 A 核心代码	18

一、 问题重述

1.1 问题背景

单波束测深利用声波在海水中的匀速直线传播测量垂直水深，其测深数据沿航迹密集、测线间缺失。多波束测深系统在与航迹垂直的平面内一次发射多个波束，在海底平坦海域能形成以测线为轴、具有一定宽度的全覆盖水深条带。条带的覆盖宽度 W 随换能器开角 θ 和水深 D 变化；相邻平行条带的重叠率定义为 $\eta = 1 - d/W$ ，其中 d 为相邻测线间距， $\eta < 0$ 表示漏测。为保证数据完整，相邻条带通常要求 10% ~ 20% 的重叠率。真实海底起伏较大，若按平均水深或最浅处水深固定布设测线，会在浅水处漏测或深水处冗余。本文研究对象与任务均来自竞赛题面 [1]。

1.2 问题内容

问题一：在与测线方向垂直的平面内，海底坡面与水平面夹角为 α 。建立多波束覆盖宽度及相邻条带重叠率的数学模型；当开角为 120° 、坡度为 1.5° 、中心水深为 70 m 时，计算表 1 所列位置的指标并保存至 **result1.xlsx**。

问题二：考虑矩形待测海域，测线方向与海底坡面法向在水平面投影的夹角为 β ，建立覆盖宽度模型；当开角为 120° 、坡度为 1.5° 、中心水深为 120 m 时，计算表 2 所列位置与方向下的覆盖宽度并保存至 **result2.xlsx**。

问题三：在南北长 2 海里、东西宽 4 海里的矩形海域内，中心水深 110 m、西深东浅、坡度 1.5° 、开角 120° 。设计一组总长度最短、完全覆盖且相邻重叠率满足 10% ~ 20% 的测线。

问题四：附件 附件.xlsx 为某海域（南北 5 海里、东西 4 海里）的单波束测深数据。设计多波束测线，使条带尽可能覆盖整个海域、相邻重叠率尽量不超过 20%、测线总长度尽可能短，并计算测线总长度、漏测面积占比与重叠率超过 20% 部分的总长度。

二、 问题分析

2.1 总体分析

四个子问题围绕“覆盖宽度”与“测线布设”两个主题层层递进：问题一建立二维剖面的基础几何模型，问题二将其推广到三维坡面，问题三在规则坡面海域上做最短测线优化，问题四进一步面向真实水深网格做数据驱动的分带布线。各问的覆盖宽度模型保持一致口径，即水平投影覆盖宽度，从而可与水平测线间距直接比较。

2.2 问题一的分析

问题一的核心是由换能器开角、坡度和测线位置确定单个条带的覆盖宽度及相邻条带的重叠率。由于题目给定的是理想斜坡剖面，所有参数已知，本问本质是二维几何建模与确定性计算。测线两侧的声束与坡面的交点不对称，左右半覆盖宽度不相等，因此重叠率不能直接套用平底公式 $\eta = 1 - d/W$ ，而应先计算相邻条带的代数重叠宽度，再除以当前条带覆盖宽度。

2.3 问题二的分析

问题二把二维剖面扩展到三维矩形海域。测线方向改变会影响两个量：船位处的水深，以及垂直测线剖面内的等效坡度。把坡面法向在水平面的投影方向与测线方向的夹角记为 β ，可将三维问题通过向量投影转化为二维剖面问题，复用问题一的覆盖宽度公式。

2.4 问题三的分析

问题三的待求量是一组测线位置，目标是总长度最短，约束是完全覆盖和重叠率在 10% ~ 20%。在单一平面坡面上，等深线为南北向直线，沿等深线布设时每条测线上的水深恒定，覆盖宽度仅随东西坐标变化，问题可降为一维布线；每条线贯穿矩形后长度固定，总长度最小等价于测线数最少。因此采用贪心递推，每步取最大可行间距，即令相邻重叠率取下限 10%。

2.5 问题四的分析

问题四的水深不再是理想平面，而是规则网格数据。由于主方案采用南北向测线，覆盖宽度主要受东西向局部坡度影响，故由附件水深估计东西向梯度，建立局部线性海底剖面模型。为兼顾浅水与深水差异，将海域按南北方向分带，逐带递推布设线段，并在原始网格上仿真覆盖与重叠，比较不同分带粒度后选择可执行性较好的方案。

三、 模型假设

为建立可计算模型，本文作出以下假设：

1. 声波在海水中的传播速度恒定，声线在均匀介质中沿直线传播，忽略折射、散射与声速剖面变化的影响；
2. 多波束换能器在同一垂直于航迹的平面内对称发射，半开角为 $\phi = \theta/2$ ；

3. 问题一至问题三中海底为理想平面斜坡，坡度 α 恒定且较小，水深沿水平方向线性变化；
4. 测线为直线，问题三中测线沿等深线方向（南北向）布设并贯穿整个矩形海域；
5. 覆盖宽度采用条带在水平面上的投影宽度，与测线水平间距保持同一量纲；
6. 附件水深数据为规则网格、无缺失值，可作为局部坡面估计与覆盖仿真依据；
7. 问题四中仅统计测量线段长度，不考虑不同线段之间的转场、调头和航行连接代价。

四、 符号说明

本文主要符号说明如表 1 所示。

表 1: 主要符号说明

符号	含义	单位
x	测线距中心点或东西向坐标	m
y	南北向坐标	m
D	海水深度	m
D_0	海域中心点水深	m
θ	换能器开角	$^\circ$ 或 rad
ϕ	半开角, $\phi = \theta/2$	$^\circ$ 或 rad
α	海底坡度	$^\circ$ 或 rad
β	测线方向与坡面法向水平投影的夹角	$^\circ$
α_{eff}	垂直测线剖面内的等效坡度	rad
B_L, B_R	左右半覆盖宽度的水平投影	m
W	条带覆盖宽度（水平投影）	m
d	相邻测线水平间距	m
O	相邻条带代数重叠宽度	m
η	相邻条带重叠率	%

五、 问题一的模型建立与求解

5.1 建模思路

以海域中心点为原点，在与测线方向垂直的二维剖面内建立水平坐标 x ，取 $x > 0$ 为浅水上坡方向、 $x < 0$ 为深水下坡方向，垂直向下为水深正方向。设换能器位于测线位置 x 处，半开角为 $\phi = \theta/2$ 。声束左右边界与斜坡海底相交，形成左右两个半覆盖宽度，其水平投影之和即为条带覆盖宽度 $W(x)$ ，如图 1 所示。

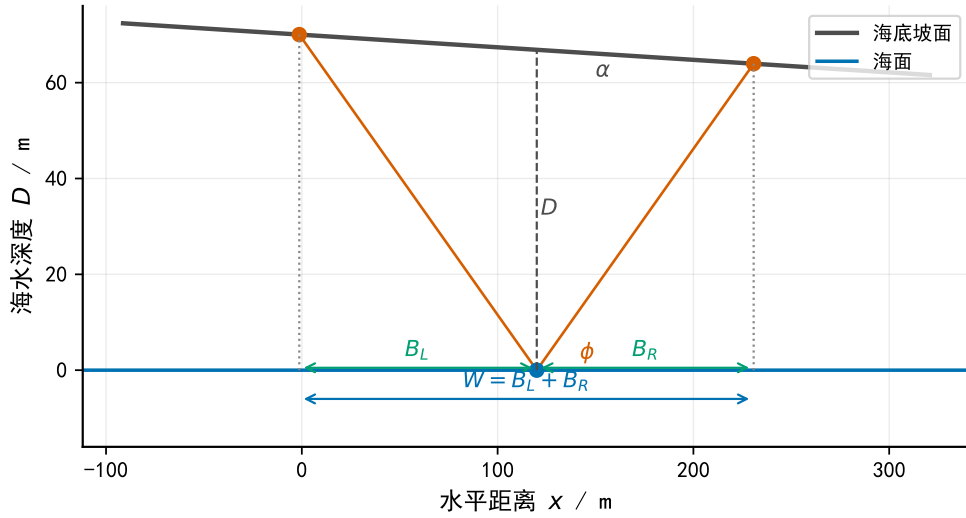


图 1: 二维斜坡剖面的覆盖宽度几何示意

5.2 水深模型

由坡度定义，位置 x 处的海水深度沿水平方向线性变化：

$$D(x) = D_0 - x \tan \alpha. \quad (1)$$

5.3 覆盖宽度模型

设声束边界与坡面的交点分别位于测线左右两侧。由正弦定理或射线与直线交点几何关系，可得左右半覆盖宽度的水平投影为

$$B_L(x) = \frac{D(x) \sin \phi \cos \alpha}{\cos(\phi + \alpha)}, \quad B_R(x) = \frac{D(x) \sin \phi \cos \alpha}{\cos(\phi - \alpha)}. \quad (2)$$

条带覆盖宽度的水平投影为

$$W(x) = B_L(x) + B_R(x) = D(x) \sin \phi \cos \alpha \left[\frac{1}{\cos(\phi + \alpha)} + \frac{1}{\cos(\phi - \alpha)} \right]. \quad (3)$$

5.4 相邻条带重叠率模型

设第 $i - 1$ 条测线与第 i 条测线的水平间距为 $d_i = x_i - x_{i-1}$ 。相邻条带的重叠宽度取两条带相交的水平宽度

$$O_i = B_R(x_{i-1}) + B_L(x_i) - d_i. \quad (4)$$

重叠率采用“相对当前条带覆盖宽度”的口径，并保留负值以表示漏测：

$$\eta_i = \frac{O_i}{W(x_i)} \times 100\%. \quad (5)$$

当 $\alpha = 0$ 时，式 (2) 退化为 $B_L = B_R = D \tan \phi$ ，进而 $W = 2D \tan(\theta/2)$ 、 $\eta = 1 - d/W$ ，与题面给出的平底公式一致。

5.5 求解与结果

取 $\theta = 120^\circ$ 、 $\alpha = 1.5^\circ$ 、 $D_0 = 70 \text{ m}$ ，对表 1 所列位置代入式 (1)–(5)，结果如表 2 所示，趋势如图 2 所示。

表 2: 问题一计算结果

测线距中心点距离 / m	−800	−600	−400	−200	0	200	400	600	800
海水深度 / m	90.95	85.71	80.47	75.24	70.00	64.76	59.53	54.29	49.05
覆盖宽度 / m	315.71	297.53	279.35	261.17	242.99	224.81	206.63	188.45	170.27
重叠率 / %	−	35.70	31.51	26.74	21.26	14.89	7.41	−1.53	−12.36

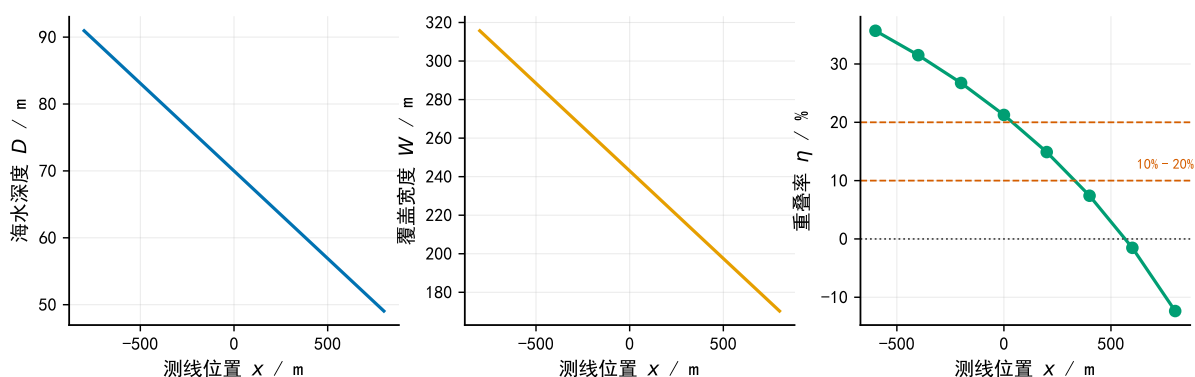


图 2: 海水深度、覆盖宽度与重叠率随测线位置的变化

由图 2 可知, 随 x 由 -800 m 增至 800 m , 水深和覆盖宽度单调减小, 符合浅水处覆盖宽度较小的物理直觉; 相邻重叠率也从 35.70% 持续下降至 -12.36% , 在右侧浅水区出现漏测。

5.6 模型验证与本问结论

模型在 $\alpha = 0$ 时精确退化为平底公式, 说明式 (2) 的几何口径正确。综合表 2, 问题一的覆盖宽度与重叠率模型已建立; 给定参数下, 覆盖宽度范围为 $170.27 \sim 315.71\text{ m}$, 且右侧浅水区会由重叠逐步转为漏测。

六、 问题二的模型建立与求解

6.1 三维坡面到二维剖面的转化

问题二把问题一的二维剖面推广到三维矩形海域。设测线方向与海底坡面法向在水平面上的投影夹角为 β , 将坡面法向水平投影的正方向取为深水方向。多波束扫描平面垂直于测线方向, 真正影响覆盖宽度的是坡度在该垂直剖面方向上的分量。因此定义垂直测线剖面内的等效坡度

$$\alpha_{\text{eff}} = \arctan(\tan \alpha \sin \beta). \quad (6)$$

船位处水深由测线方向上的坡面高程变化确定。设测量船距海域中心点的距离为 r (海里), 换算为 $s = 1852r$ 米后,

$$D(r, \beta) = D_0 + s \tan \alpha \cos \beta. \quad (7)$$

将问题一公式中的坡度替换为 α_{eff} 、水深替换为 $D(r, \beta)$, 得到覆盖宽度

$$W(r, \beta) = D(r, \beta) \sin \phi \cos \alpha_{\text{eff}} \left[\frac{1}{\cos(\phi + \alpha_{\text{eff}})} + \frac{1}{\cos(\phi - \alpha_{\text{eff}})} \right]. \quad (8)$$

6.2 求解与结果

取 $\theta = 120^\circ$ 、 $\alpha = 1.5^\circ$ 、 $D_0 = 120\text{ m}$, 对 $\beta = 0^\circ, 45^\circ, \dots, 315^\circ$ 与 $r = 0, 0.3, \dots, 2.1$ 海里代入式 (7)–(8), 结果如表 3 和图 3 所示。

表 3: 问题二覆盖宽度 / m

$\beta / ^\circ$	0	0.3	0.6	0.9	1.2	1.5	1.8	2.1
0	415.69	466.09	516.49	566.89	617.29	667.69	718.09	768.48
45	416.12	451.79	487.47	523.14	558.82	594.49	630.16	665.84
90	416.55	416.55	416.55	416.55	416.55	416.55	416.55	416.55
135	416.12	380.45	344.77	309.10	273.42	237.75	202.08	166.40
180	415.69	365.29	314.89	264.50	214.10	163.70	113.30	62.90
225	416.12	380.45	344.77	309.10	273.42	237.75	202.08	166.40
270	416.55	416.55	416.55	416.55	416.55	416.55	416.55	416.55
315	416.12	451.79	487.47	523.14	558.82	594.49	630.16	665.84

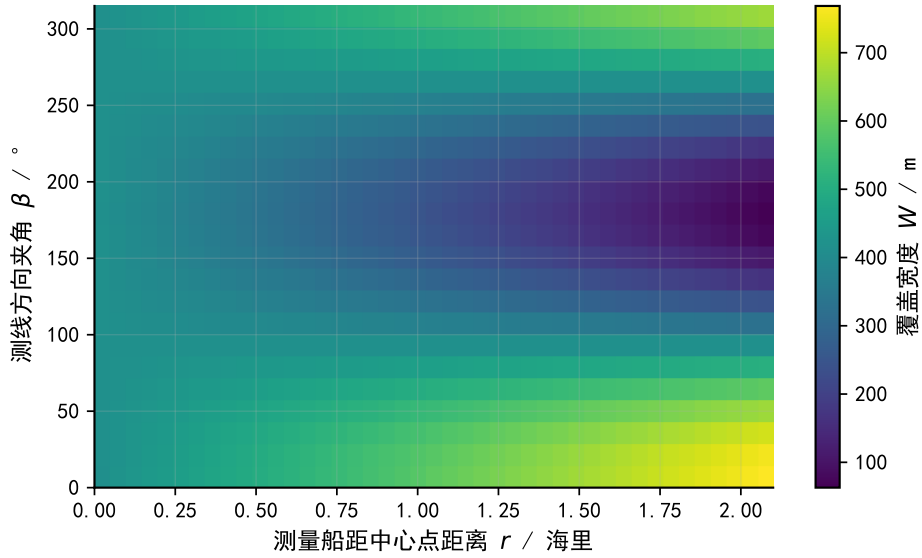


图 3: 覆盖宽度随测线方向夹角与船位距离的变化

6.3 模型验证与本问结论

当 $\beta = 0^\circ$ 或 180° 时, 测线方向沿坡面法向水平投影, 垂直测线剖面沿等深线方向, $\alpha_{\text{eff}} = 0$, 公式退化为平底公式 $W = 2D \tan(\theta/2)$; 当 $\beta = 90^\circ$ 或 270° 时, 船沿等深线运动, $D(r, \beta) = D_0$, 覆盖宽度不随距离变化, 与表 3 中该行恒为 416.55 m 一致。整体上覆盖宽度随水深增大而增大, $\beta = 0^\circ$ 行随距离显著增大、 $\beta = 180^\circ$ 行随距离显著减小, 符合物理直觉。

七、 问题三的建立与求解

7.1 优化模型的建立

以矩形海域中心为原点建立水平坐标系， x 向东为正（东侧为浅水）， y 向北为正。海域东西边界为 $x_{\min} = -3704 \text{ m}$ 、 $x_{\max} = 3704 \text{ m}$ ，南北边界为 $y_{\min} = -1852 \text{ m}$ 、 $y_{\max} = 1852 \text{ m}$ （1 海里 = 1852 米）。

由于海底为西深东浅的单一平面，等深线为南北向直线。测线沿等深线南北向布设时，每条线上的水深恒定，问题可降为一维布线。设第 i 条测线坐标为 x_i ，其两侧水平半覆盖宽度为

$$B_W(x_i) = \frac{D(x_i) \sin \phi \cos \alpha}{\cos(\phi + \alpha)}, \quad B_E(x_i) = \frac{D(x_i) \sin \phi \cos \alpha}{\cos(\phi - \alpha)}, \quad (9)$$

其中 $D(x_i) = 110 - x_i \tan \alpha$ 。相邻条带的代数重叠宽度与重叠率为

$$O_i = B_E(x_{i-1}) + B_W(x_i) - (x_i - x_{i-1}), \quad \eta_i = \frac{O_i}{B_W(x_i) + B_E(x_i)} \times 100\%. \quad (10)$$

为保证完全覆盖，首条测线西侧覆盖边界应覆盖西边界，末条测线东侧覆盖边界应覆盖东边界：

$$x_1 - B_W(x_1) \leq x_{\min}, \quad x_n + B_E(x_n) \geq x_{\max}. \quad (11)$$

相邻重叠率应满足 $10\% \leq \eta_i \leq 20\%$ 。每条测线贯穿矩形，长度均为 $y_{\max} - y_{\min} = 3704 \text{ m}$ ，故总长度最小等价于测线数最少。问题三转化为

$$\begin{aligned} \min n, \quad \text{s.t.} \quad & 10\% \leq \eta_i \leq 20\%, \quad i = 2, \dots, n, \\ & x_1 - B_W(x_1) \leq x_{\min}, \\ & x_n + B_E(x_n) \geq x_{\max}. \end{aligned} \quad (12)$$

7.2 贪心递推求解

为最小化测线数，第一步将首条测线尽可能向东放置，同时令其西侧覆盖边界正好覆盖西边界，即求解 $x_1 - B_W(x_1) = x_{\min}$ 。此后每一步在保证与前一条测线之间无漏测的前提下取最大可行间距，即令重叠率取约束下限 $\eta_i = 10\%$ ，用二分法递推 x_i ，直至末条测线覆盖东边界。

7.3 求解结果

程序得到推荐方案：测线方向为南北向、平行等深线，共 34 条；每条测线长 2 海里，总长度 68.00 海里，即 125936.00 m。布设俯视图见图 4，相邻间距与重叠率校验见图 5。

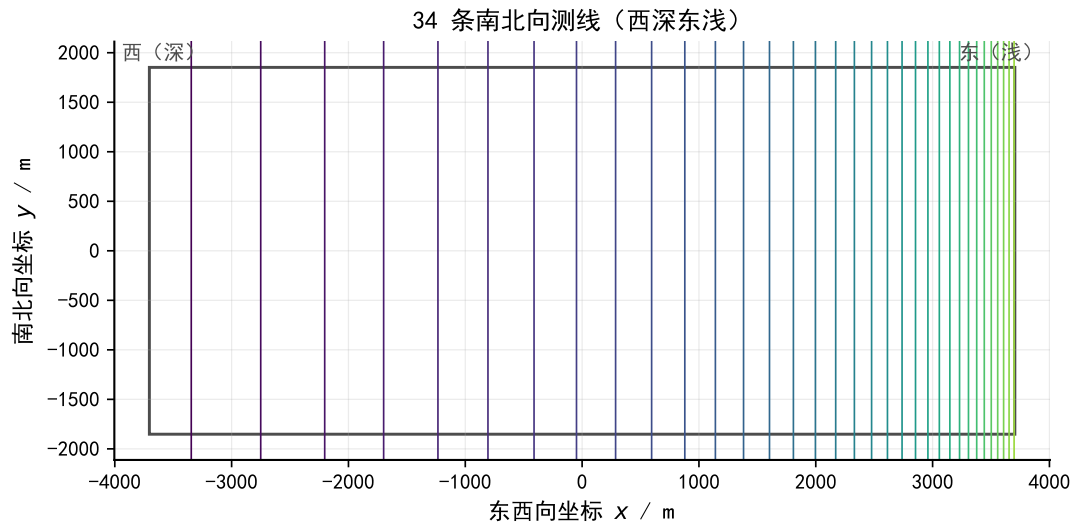


图 4: 34 条南北向测线在矩形海域内的布设

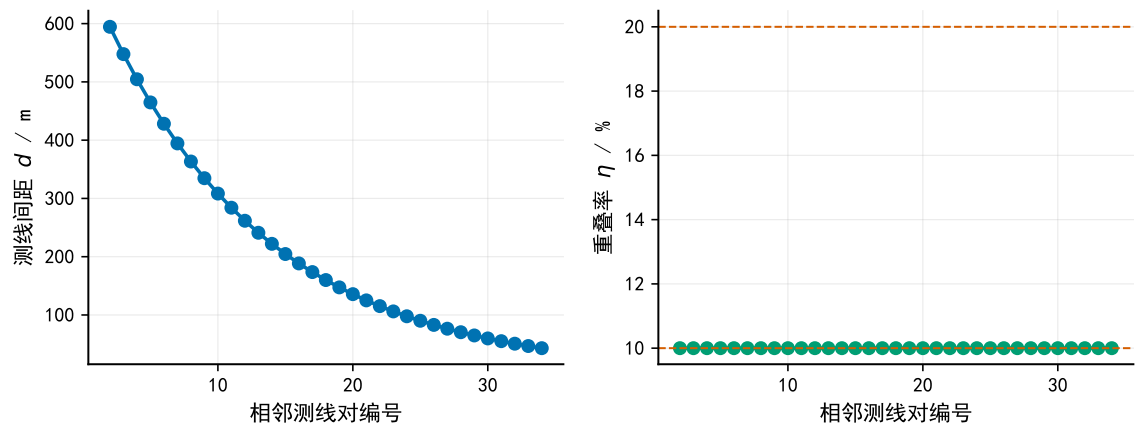


图 5: 相邻测线间距与重叠率随测线对编号的变化

7.4 模型验证与本问结论

完全覆盖检验表明，首条测线西覆盖边界为 -3704.00 m 、末条测线东覆盖边界为 3719.41 m ，东西方向无漏测；逐对复算 33 个相邻重叠率均为 10.00% ，满足 $10\% \sim 20\%$ 。最小条数检验表明，若少一条测线，在同样取最大可行间距的情况下最东覆盖边界仅为 3678.24 m ，距东边界仍有 25.76 m 缺口，因此 33 条不可行。故该方案在所选方向与布线规则下测线数已不可再减少。

八、 问题四的模型建立与求解

8.1 数据说明与局部坡度估计

附件给出海域（南北 5 海里、东西 4 海里）的规则水深网格：横向 201 个点（0 ~ 4 海里）、纵向 251 个点（0 ~ 5 海里），步长均为 0.02 海里，水深点共 50451 个，无缺失，水深范围为 20.00 ~ 197.20 m，平均水深 62.54 m。数据空间分布见图 6。

由于主方案采用南北向测线，覆盖宽度主要受东西向剖面坡度影响，故由水深场估计东西向局部梯度 $g_x = \partial D / \partial x$ 。对南北向测线，将局部海底剖面在测线两侧近似为

$$D(x+u, y) \approx D(x, y) + g_x(x, y)u. \quad (13)$$

声束边界与局部线性海底相交，得到两侧水平半覆盖宽度

$$B_W(x, y) = \frac{D(x, y)}{\cot \phi + g_x(x, y)}, \quad B_E(x, y) = \frac{D(x, y)}{\cot \phi - g_x(x, y)}. \quad (14)$$

条带水平覆盖宽度为 $W_h = B_W + B_E$ 。相邻两条测线 $x_i < x_{i+1}$ 在同一纵向位置 y 的重叠率为

$$\eta_i(y) = \frac{B_E(x_i, y) + B_W(x_{i+1}, y) - (x_{i+1} - x_i)}{W_h(x_{i+1}, y)} \times 100\%. \quad (15)$$

网格点 (x, y) 被第 i 条测线覆盖的条件为 $x_i - B_W(x_i, y) \leq x \leq x_i + B_E(x_i, y)$ 。漏测率用未被覆盖的网格点数占总网格点数的百分比近似。

8.2 分带递推布线与覆盖评价

真实水深在南北向存在变化，若全海域统一布设南北向长线，深浅差异会造成大量冗余。本文按南北方向把海域分成若干水平带，在每带内采用与问题三相同的递推法布设南北向线段：首条线尽量靠东且覆盖西边界，后续线在保证无漏测的前提下取最大间距，直到覆盖东边界。合并各带线段后，在原始网格上逐点复算覆盖与重叠指标。

8.3 候选方案比较与推荐方案

分别取分带高度 5.00、1.00、0.50、0.25 海里进行布设，四项指标如表 4 与图 7 所示。

表 4: 问题四候选方案比较

方案	线段数	总长度 / m	漏测率 / %	超 20% 重叠长度 / m	最大重叠率 / %
全域 5.00 NM	60	555600.00	0.00	276877.69	76.42
1.00 NM	240	444480.00	0.00	51525.55	45.86
0.50 NM (推荐)	461	426886.00	0.00	5481.92	49.01
0.25 NM	897	415311.00	0.00	2742.38	46.07

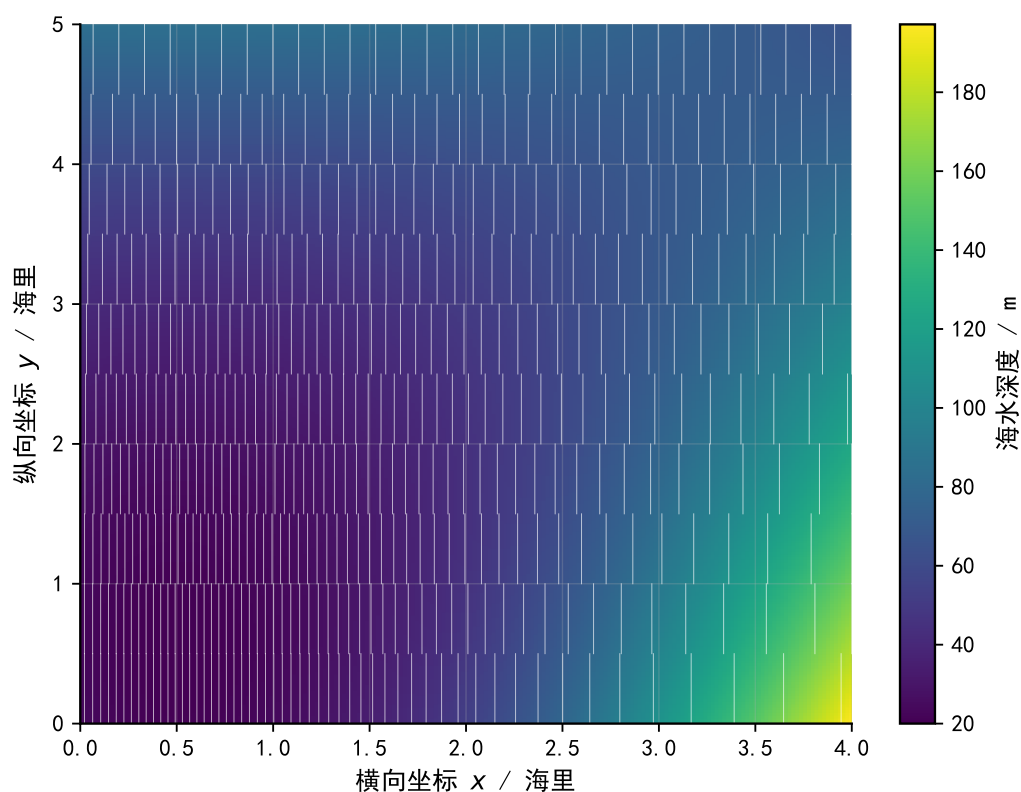


图 6: 附件水深空间分布与 0.50 NM 推荐测线叠加

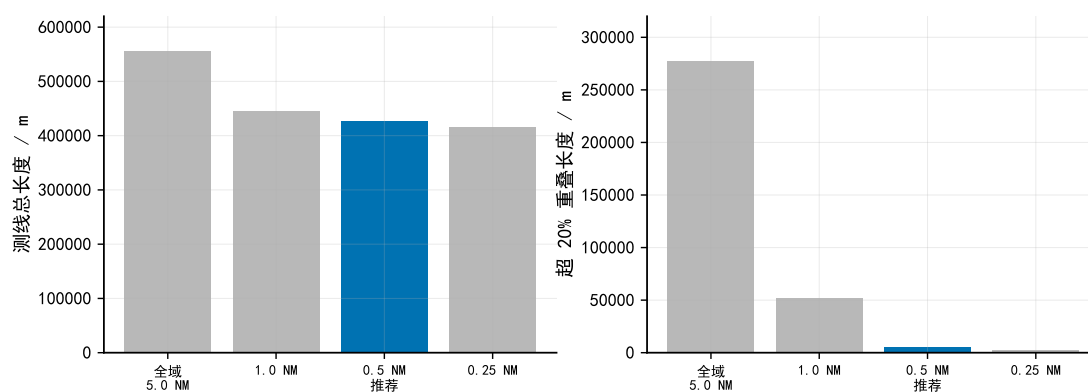


图 7: 四种分带方案的测线总长度与超 20% 重叠长度比较

本文推荐 0.50 海里分带方案。该方案在漏测率为 0.00% 的前提下，把重叠率超过 20% 的部分总长度压至 5481.92 m（2.96 海里），测线总长度 426886.00 m（230.50 海里），平均相邻重叠率 4.66%。0.25 海里方案表内指标虽略优，但线段数由 461 增至 897，转场与调头成本显著增加，而题目指标未计入该部分代价，故不作为主推荐。

8.4 模型验证与本问结论

数据完整性检验表明附件网格规模、坐标范围与题面一致且无缺失；覆盖率复算给出漏测率 0.00%，说明推荐方案完全覆盖待测海域；分带敏感性显示，由全域统一布线到 0.50 海里分带，总长度由 555600.00 m 降至 426886.00 m，超 20% 重叠长度由 276877.69 m 降至 5481.92 m，说明分带策略有效。综上，推荐方案能兼顾完全覆盖、冗余控制和可执行性。

九、 敏感性分析

9.1 问题一覆盖宽度对坡度与开角的敏感性

为考察几何模型对关键输入参数的敏感程度，在 $x = 0$ 、 $D_0 = 70$ m 处分别单独扰动坡度 α 与换能器开角 θ ，覆盖宽度变化如表 5 所示。

表 5: 问题一覆盖宽度对坡度与开角的单因素敏感性

$\alpha / ^\circ$ ($\theta = 120^\circ$)	0.0	0.5	1.0	1.5	2.0	3.0
W / m	242.49	242.54	242.71	242.99	243.38	244.50
$\theta / ^\circ$ ($\alpha = 1.5^\circ$)	100	110	120	130	140	—
W / m	167.01	200.22	242.99	301.18	386.65	—

由表 5 可知，覆盖宽度对坡度 α 的敏感程度较低，在 $0.0^\circ \sim 3.0^\circ$ 范围内 W 仅变化约 2.01 m；而覆盖宽度对开角 θ 明显更敏感， θ 从 100° 增至 140° 时 W 由 167.01 m 增至 386.65 m。这说明在本题给定的小坡度范围内，可把坡度近似作为弱影响因素，而换能器开角是决定覆盖宽度的主要参数。

9.2 问题三最小测线数的约束敏感性

问题三的递推求解取重叠率下限 10% 作为每步最大间距条件，方案恰好落在约束边界上。若在相同布线方向下减少一条测线，最东覆盖边界由 3719.41 m 降至 3678.24 m，出现 25.76 m 缺口。该结果表明 34 条测线对该 10% ~ 20% 重叠率约束是紧的，方案对

重叠率下限的变化较为敏感。

9.3 问题四分带粒度的敏感性

问题四的分带高度从 5.00 海里细化到 0.25 海里时，测线总长度由 555600.00 m 降至 415311.00 m，超 20% 重叠长度由 276877.69 m 降至 2742.38 m，漏测率始终为 0.00%。可见分带粒度对冗余控制影响显著；推荐 0.50 海里方案是在三项指标与线段数、转场成本之间的折中选择。

十、 模型评价与推广

10.1 模型优点

1. 覆盖宽度与重叠率全程使用“水平投影”口径，可与水平测线间距直接比较，避免了斜坡情形下坡面长度与水平距离混用的问题；
2. 问题一、二均为确定性解析几何模型，参数可解释、结果可复算，并在 $\alpha = 0$ 或 $\beta = 0^\circ, 180^\circ$ 等特殊情形下退化为平底公式，模型正确性有明确检验；
3. 问题三的贪心递推针对单一平面坡面可证明在当前方向下测线数不可再减少，结果具有可解释的最优性；
4. 问题四采用分带策略并保留网格覆盖仿真，能够处理真实地形下的深浅差异，方案与题目三项指标直接对应。

10.2 模型不足

1. 问题一至问题三假设海底为理想平面斜坡、声波沿直线传播，未考虑真实声速剖面、折射和复杂地形；
2. 问题四仅统计测量线段长度，未计入不同线段之间的转场、调头和航行连接成本，可能低估实际作业代价；
3. 分带边界处各带独立布线的覆盖近似会引入少量重叠误差，局部最大相邻重叠率仍达 49.01%；
4. 问题四未进行随机扰动或全局优化，所得为当前分带与递推规则下的较优方案，而非严格全局最优。

10.3 模型推广

模型可推广到一般起伏海底：保留“水平投影覆盖宽度 + 相邻重叠率”的评价框架，将平面坡度替换为局部梯度估计即可用于非规则地形。若进一步引入声速分层，可在射线追踪中修正声线曲率；若考虑船舶转弯与多船协同，可把分带递推扩展为带转场代价的路径优化问题。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2023 年高教社杯全国大学生数学建模竞赛 B 题: 多波束测线问题. 2023.

附录 A 核心代码

以下给出问题一与问题三的核心程序。完整程序、运行说明与结果文件见项目仓库 `code/` 与 `result1-4.xlsx`。

```
"""Question 1 multibeam swath-width model for CUMCM 2023B.

The script computes the depth, swath coverage width, and overlap rate
for the locations requested in Table 1, then writes result1.xlsx.
"""

from __future__ import annotations

import csv
import math
from dataclasses import dataclass
from pathlib import Path

from openpyxl import Workbook
from openpyxl.styles import Alignment, Font, PatternFill
from openpyxl.utils import get_column_letter

ROOT = Path(__file__).resolve().parents[1]
RESULT_XLSX = ROOT / "result1.xlsx"
RESULT_CSV = ROOT / "docs" / "q1_result_table.csv"

THETA_DEG = 120.0
ALPHA_DEG = 1.5
CENTER_DEPTH_M = 70.0
POSITIONS_M = [-800, -600, -400, -200, 0, 200, 400, 600, 800]

@dataclass(frozen=True)
class Row:
    x_m: float
```

```

depth_m: float
left_half_width_m: float
right_half_width_m: float
coverage_width_m: float
overlap_width_m: float | None
overlap_rate_pct: float | None
simplified_overlap_rate_pct: float | None

def depth_at(x_m: float, center_depth_m: float, alpha_rad: float) -> float:
    """Depth at horizontal offset x; positive x is the shallower upslope
    side."""
    return center_depth_m - x_m * math.tan(alpha_rad)

def half_widths(depth_m: float, theta_rad: float, alpha_rad: float) ->
    tuple[float, float]:
    """Return horizontal left and right half-widths of one swath strip.

    The seabed is approximated by a straight slope in the cross-track plane
    .
    The left half is toward the deeper side when x is ordered from negative
    to
    positive. Both widths are horizontal projections, so they are
    comparable
    with the horizontal line spacing in the table.
    """
    half_angle = theta_rad / 2.0
    common = depth_m * math.sin(half_angle) * math.cos(alpha_rad)
    left = common / math.cos(half_angle + alpha_rad)
    right = common / math.cos(half_angle - alpha_rad)
    return left, right

def compute_rows() -> list[Row]:
    theta_rad = math.radians(THETA_DEG)

```

```

alpha_rad = math.radians(ALPHA_DEG)

rows: list[Row] = []
previous: Row | None = None
for x_m in POSITIONS_M:
    depth_m = depth_at(x_m, CENTER_DEPTH_M, alpha_rad)
    left_half, right_half = half_widths(depth_m, theta_rad, alpha_rad)
    coverage = left_half + right_half

    overlap_width = None
    overlap_rate = None
    simplified_overlap_rate = None
    if previous is not None:
        spacing = x_m - previous.x_m
        overlap_width = previous.right_half_width_m + left_half -
spacing
        overlap_rate = overlap_width / coverage * 100.0
        simplified_overlap_rate = (1.0 - spacing / coverage) * 100.0

    rows.append(
        Row(
            x_m=x_m,
            depth_m=depth_m,
            left_half_width_m=left_half,
            right_half_width_m=right_half,
            coverage_width_m=coverage,
            overlap_width_m=overlap_width,
            overlap_rate_pct=overlap_rate,
            simplified_overlap_rate_pct=simplified_overlap_rate,
        )
    )
    previous = rows[-1]

return rows

```

```

def assert_flat_limit() -> None:
    """Check the model reduces to the textbook flat-bottom formula."""
    theta_rad = math.radians(THETA_DEG)
    flat_left, flat_right = half_widths(CENTER_DEPTH_M, theta_rad, 0.0)
    flat_width = flat_left + flat_right
    expected = 2.0 * CENTER_DEPTH_M * math.tan(theta_rad / 2.0)
    if not math.isclose(flat_width, expected, rel_tol=1e-12, abs_tol=1e-12):
        raise AssertionError((flat_width, expected))

def write_csv(rows: list[Row]) -> None:
    RESULT_CSV.parent.mkdir(parents=True, exist_ok=True)
    with RESULT_CSV.open("w", newline="", encoding="utf-8-sig") as f:
        writer = csv.writer(f)
        writer.writerow(["指标"] + [int(row.x_m) for row in rows])
        writer.writerow(["海水深度/m"] + [f"{row.depth_m:.2f}" for row in rows])
        writer.writerow(["覆盖宽度/m"] + [f"{row.coverage_width_m:.2f}" for row in rows])
        writer.writerow(
            ["与前一条测线的重叠率/%"]
            + ["--" if row.overlap_rate_pct is None else f"{row.overlap_rate_pct:.2f}" for row in rows]
        )

def write_workbook(rows: list[Row]) -> None:
    wb = Workbook()
    ws = wb.active
    ws.title = "问题1结果"

    table = [
        ["测线距中心点处的距离/m"] + [int(row.x_m) for row in rows],
        ["海水深度/m"] + [round(row.depth_m, 2) for row in rows],
        ["覆盖宽度/m"] + [round(row.coverage_width_m, 2) for row in rows],
    ]

```

```

        [
            "与前一条测线的重叠率/%"
        ]
        + ["--" if row.overlap_rate_pct is None else round(row.
overlap_rate_pct, 2) for row in rows],
    ]

    for r_idx, row_values in enumerate(table, start=1):
        for c_idx, value in enumerate(row_values, start=1):
            cell = ws.cell(r_idx, c_idx, value)
            cell.alignment = Alignment(horizontal="center", vertical="
center")
            if r_idx == 1 or c_idx == 1:
                cell.font = Font(bold=True)
                cell.fill = PatternFill("solid", fgColor="D9EAF7")

ws.column_dimensions["A"].width = 28
for c_idx in range(2, len(rows) + 2):
    ws.column_dimensions[get_column_letter(c_idx)].width = 12

detail = wb.create_sheet("计算明细")
headers = [
    "x/m",
    "D/m",
    "左半宽/m",
    "右半宽/m",
    "覆盖宽度/m",
    "重叠宽度/m",
    "精确重叠率/%",
    "平底简化重叠率/%(备用)",
]
detail.append(headers)
for cell in detail[1]:
    cell.font = Font(bold=True)
    cell.fill = PatternFill("solid", fgColor="E2F0D9")
    cell.alignment = Alignment(horizontal="center", vertical="center")

```

```

for row in rows:
    detail.append(
        [
            row.x_m,
            row.depth_m,
            row.left_half_width_m,
            row.right_half_width_m,
            row.coverage_width_m,
            row.overlap_width_m,
            row.overlap_rate_pct,
            row.simplified_overlap_rate_pct,
        ]
    )

for column in detail.columns:
    detail.column_dimensions[column[0].column_letter].width = 18
    for cell in column:
        cell.alignment = Alignment(horizontal="center", vertical="center")
        if isinstance(cell.value, float):
            cell.number_format = "0.00"

wb.save(RESULT_XLSX)

def print_table(rows: list[Row]) -> None:
    print("Table 1 results, exact overlap-rate definition")
    print(["指标"] + [int(row.x_m) for row in rows])
    print(["海水深度/m"] + [round(row.depth_m, 2) for row in rows])
    print(["覆盖宽度/m"] + [round(row.coverage_width_m, 2) for row in rows])
    print(
        ["与前一条测线的重叠率/%"]
        + ["--" if row.overlap_rate_pct is None else round(row.
overlap_rate_pct, 2) for row in rows]
    )

```



```

def main() -> None:
    assert_flat_limit()
    rows = compute_rows()
    write_csv(rows)
    write_workbook(rows)
    print_table(rows)
    print(f"Wrote {RESULT_XLSX}")
    print(f"Wrote {RESULT_CSV}")

if __name__ == "__main__":
    main()

```

```

"""Question 3 survey-line design for CUMCM 2023B.

```

The script designs north-south survey lines parallel to depth contours in the simple rectangular slope area, verifies full coverage and adjacent overlap constraints, and writes result3.xlsx.

```

"""

```

```

from __future__ import annotations

```

```

import math

```

```

from dataclasses import dataclass

```

```

from pathlib import Path

```

```

from openpyxl import Workbook

```

```

from openpyxl.styles import Alignment, Font, PatternFill

```

```

from openpyxl.utils import get_column_letter

```

```

ROOT = Path(__file__).resolve().parents[1]

```

```

RESULT_XLSX = ROOT / "result3.xlsx"

```

```

NAUTICAL_MILE_M = 1852.0
THETA_DEG = 120.0
ALPHA_DEG = 1.5
CENTER_DEPTH_M = 110.0
EAST_WEST_WIDTH_NM = 4.0
NORTH_SOUTH_LENGTH_NM = 2.0
MIN_OVERLAP_RATE = 0.10
MAX_OVERLAP_RATE = 0.20
TARGET_OVERLAP_RATE = MIN_OVERLAP_RATE

X_MIN_M = -EAST_WEST_WIDTH_NM * NAUTICAL_MILE_M / 2.0
X_MAX_M = EAST_WEST_WIDTH_NM * NAUTICAL_MILE_M / 2.0
Y_MIN_M = -NORTH_SOUTH_LENGTH_NM * NAUTICAL_MILE_M / 2.0
Y_MAX_M = NORTH_SOUTH_LENGTH_NM * NAUTICAL_MILE_M / 2.0
LINE_LENGTH_M = Y_MAX_M - Y_MIN_M

@dataclass(frozen=True)
class SurveyLine:
    index: int
    x_m: float
    y_start_m: float
    y_end_m: float
    depth_m: float
    west_half_width_m: float
    east_half_width_m: float
    coverage_width_m: float
    west_edge_m: float
    east_edge_m: float
    spacing_from_previous_m: float | None
    overlap_width_m: float | None
    overlap_rate_pct: float | None

def depth_at(x_m: float, center_depth_m: float, alpha_rad: float) -> float:

```

```

        """Depth at east-west coordinate x; positive x is the shallow east side
        ."""
        return center_depth_m - x_m * math.tan(alpha_rad)

def half_widths(depth_m: float, theta_rad: float, alpha_rad: float) ->
    tuple[float, float]:
    """Return horizontal west/east half-widths for a north-south survey
    line."""
    half_angle = theta_rad / 2.0
    common = depth_m * math.sin(half_angle) * math.cos(alpha_rad)
    west = common / math.cos(half_angle + alpha_rad)
    east = common / math.cos(half_angle - alpha_rad)
    return west, east

def west_edge(x_m: float, theta_rad: float, alpha_rad: float) -> float:
    west_half, _ = half_widths(depth_at(x_m, CENTER_DEPTH_M, alpha_rad),
    theta_rad, alpha_rad)
    return x_m - west_half

def east_edge(x_m: float, theta_rad: float, alpha_rad: float) -> float:
    _, east_half = half_widths(depth_at(x_m, CENTER_DEPTH_M, alpha_rad),
    theta_rad, alpha_rad)
    return x_m + east_half

def overlap_rate(previous_x_m: float, current_x_m: float, theta_rad: float,
    alpha_rad: float) -> float:
    """Adjacent overlap rate using the current line's horizontal swath
    width."""
    previous_depth = depth_at(previous_x_m, CENTER_DEPTH_M, alpha_rad)
    current_depth = depth_at(current_x_m, CENTER_DEPTH_M, alpha_rad)
    _, previous_east_half = half_widths(previous_depth, theta_rad,
    alpha_rad)

```

```

    current_west_half, current_east_half = half_widths(current_depth,
theta_rad, alpha_rad)
    current_width = current_west_half + current_east_half
    overlap_width = previous_east_half + current_west_half - (current_x_m -
previous_x_m)
    return overlap_width / current_width

def solve_first_line_x(theta_rad: float, alpha_rad: float) -> float:
    """Place the first line as far east as possible while covering the west
    edge."""
    lo = X_MIN_M - 2000.0
    hi = X_MAX_M
    for _ in range(100):
        mid = (lo + hi) / 2.0
        if west_edge(mid, theta_rad, alpha_rad) <= X_MIN_M:
            lo = mid
        else:
            hi = mid
    return lo

def solve_next_line_x(previous_x_m: float, theta_rad: float, alpha_rad:
float) -> float:
    """Maximize the next x while keeping the overlap rate at least 10%."""
    lo = previous_x_m
    hi = max(X_MAX_M, previous_x_m + 1.0)
    while overlap_rate(previous_x_m, hi, theta_rad, alpha_rad) >
TARGET_OVERLAP_RATE:
        hi += 1000.0
        if hi > X_MAX_M + 10000.0:
            raise RuntimeError("Failed to bracket the next survey line
position.")

    for _ in range(100):
        mid = (lo + hi) / 2.0

```

```

        if overlap_rate(previous_x_m, mid, theta_rad, alpha_rad) >=
TARGET_OVERLAP_RATE:
            lo = mid
        else:
            hi = mid
    return lo

def make_line(
    index: int,
    x_m: float,
    theta_rad: float,
    alpha_rad: float,
    previous: SurveyLine | None = None,
) -> SurveyLine:
    depth = depth_at(x_m, CENTER_DEPTH_M, alpha_rad)
    west_half, east_half = half_widths(depth, theta_rad, alpha_rad)
    spacing = None
    overlap_width = None
    overlap_rate_pct = None
    if previous is not None:
        spacing = x_m - previous.x_m
        overlap_width = previous.east_half_width_m + west_half - spacing
        overlap_rate_pct = overlap_width / (west_half + east_half) * 100.0

    return SurveyLine(
        index=index,
        x_m=x_m,
        y_start_m=Y_MIN_M,
        y_end_m=Y_MAX_M,
        depth_m=depth,
        west_half_width_m=west_half,
        east_half_width_m=east_half,
        coverage_width_m=west_half + east_half,
        west_edge_m=x_m - west_half,
        east_edge_m=x_m + east_half,

```

```

        spacing_from_previous_m=spacing,
        overlap_width_m=overlap_width,
        overlap_rate_pct=overlap_rate_pct,
    )

def design_lines() -> list[SurveyLine]:
    theta_rad = math.radians(THETA_DEG)
    alpha_rad = math.radians(ALPHA_DEG)

    lines: list[SurveyLine] = []
    x_m = solve_first_line_x(theta_rad, alpha_rad)
    lines.append(make_line(1, x_m, theta_rad, alpha_rad))

    while lines[-1].east_edge_m < X_MAX_M:
        x_m = solve_next_line_x(lines[-1].x_m, theta_rad, alpha_rad)
        lines.append(make_line(len(lines) + 1, x_m, theta_rad, alpha_rad,
lines[-1]))

    return lines

def assert_flat_limit() -> None:
    theta_rad = math.radians(THETA_DEG)
    flat_west, flat_east = half_widths(CENTER_DEPTH_M, theta_rad, 0.0)
    expected = CENTER_DEPTH_M * math.tan(theta_rad / 2.0)
    if not math.isclose(flat_west, expected, rel_tol=1e-12, abs_tol=1e-12):
        raise AssertionError((flat_west, expected))
    if not math.isclose(flat_east, expected, rel_tol=1e-12, abs_tol=1e-12):
        raise AssertionError((flat_east, expected))

def max_east_edge_with_line_count(count: int) -> float:
    """Best east coverage achievable by count lines under the selected
model."""
    if count < 1:

```

```

        raise ValueError("count must be positive")

    theta_rad = math.radians(THETA_DEG)
    alpha_rad = math.radians(ALPHA_DEG)
    x_m = solve_first_line_x(theta_rad, alpha_rad)
    for _ in range(count - 1):
        x_m = solve_next_line_x(x_m, theta_rad, alpha_rad)
    return east_edge(x_m, theta_rad, alpha_rad)

def validate_design(lines: list[SurveyLine]) -> dict[str, float]:
    if not lines:
        raise AssertionError("No survey lines generated.")

    tol = 1e-7
    if lines[0].west_edge_m > X_MIN_M + tol:
        raise AssertionError("The west boundary is not covered.")
    if lines[-1].east_edge_m < X_MAX_M - tol:
        raise AssertionError("The east boundary is not covered.")

    overlap_rates = [line.overlap_rate_pct for line in lines[1:]]
    for rate in overlap_rates:
        if rate is None:
            raise AssertionError("Missing overlap rate.")
        if rate < MIN_OVERLAP_RATE * 100.0 - 1e-6 or rate >
MAX_OVERLAP_RATE * 100.0 + 1e-6:
            raise AssertionError(f"Overlap rate out of range: {rate}")

    previous_count_east_edge = max_east_edge_with_line_count(len(lines) -
1)
    if previous_count_east_edge >= X_MAX_M - tol:
        raise AssertionError("The line count is not minimal for the
selected orientation.")

    return {
        "line_count": float(len(lines)),

```

```

    "total_length_m": len(lines) * LINE_LENGTH_M,
    "total_length_nm": len(lines) * NORTH_SOUTH_LENGTH_NM,
    "west_boundary_m": X_MIN_M,
    "east_boundary_m": X_MAX_M,
    "first_west_edge_m": lines[0].west_edge_m,
    "last_east_edge_m": lines[-1].east_edge_m,
    "east_excess_m": lines[-1].east_edge_m - X_MAX_M,
    "min_overlap_pct": min(overlap_rates),
    "max_overlap_pct": max(overlap_rates),
    "previous_count_east_edge_m": previous_count_east_edge,
    "previous_count_gap_m": X_MAX_M - previous_count_east_edge,
}

```

```

def write_workbook(lines: list[SurveyLine], summary: dict[str, float]) ->

```

```

    None:

```

```

    wb = Workbook()

```

```

    ws = wb.active

```

```

    ws.title = "方案摘要"

```

```

    summary_rows = [

```

```

        ("测线方向", "南北向, 平行等深线"),

```

```

        ("坐标约定", "x向东为正, y向北为正, 原点为海域中心"),

```

```

        ("东西宽度/m", EAST_WEST_WIDTH_NM * NAUTICAL_MILE_M),

```

```

        ("南北长度/m", NORTH_SOUTH_LENGTH_NM * NAUTICAL_MILE_M),

```

```

        ("换能器开角/deg", THETA_DEG),

```

```

        ("坡度/deg", ALPHA_DEG),

```

```

        ("中心水深/m", CENTER_DEPTH_M),

```

```

        ("重叠率下限/%", MIN_OVERLAP_RATE * 100.0),

```

```

        ("重叠率上限/%", MAX_OVERLAP_RATE * 100.0),

```

```

        ("设计测线数/条", int(summary["line_count"])),

```

```

        ("测线总长度/m", summary["total_length_m"]),

```

```

        ("测线总长度/NM", summary["total_length_nm"]),

```

```

        ("最小相邻重叠率/%", summary["min_overlap_pct"]),

```

```

        ("最大相邻重叠率/%", summary["max_overlap_pct"]),

```

```

        ("西边界/m", summary["west_boundary_m"]),

```



```

        ("首条测线西覆盖边界/m", summary["first_west_edge_m"]),
        ("东边界/m", summary["east_boundary_m"]),
        ("末条测线东覆盖边界/m", summary["last_east_edge_m"]),
        ("东侧超出覆盖/m", summary["east_excess_m"]),
        ("若少一条线的最东覆盖边界/m", summary["previous_count_east_edge_m"])
    ,
        ("若少一条线的东侧缺口/m", summary["previous_count_gap_m"]),
    ]
ws.append(["项目", "数值"])
for row in summary_rows:
    ws.append(list(row))

ws["A1"].font = Font(bold=True)
ws["B1"].font = Font(bold=True)
for row in ws.iter_rows():
    for cell in row:
        cell.alignment = Alignment(horizontal="center", vertical="center")
        if cell.row == 1:
            cell.fill = PatternFill("solid", fgColor="D9EAF7")
ws.column_dimensions["A"].width = 30
ws.column_dimensions["B"].width = 34

detail = wb.create_sheet("测线坐标与覆盖")
detail_headers = [
    "line_id",
    "x/m",
    "x/NM",
    "y_start/m",
    "y_start/NM",
    "y_end/m",
    "y_end/NM",
    "line_length/m",
    "D/m",
    "west_half_width/m",
    "east_half_width/m",

```

```

        "W_h/m",
        "west_edge/m",
        "east_edge/m",
        "spacing_from_previous/m",
        "overlap_width/m",
        "overlap_rate/%",
    ]
    detail.append(detail_headers)
    for line in lines:
        detail.append(
            [
                line.index,
                line.x_m,
                line.x_m / NAUTICAL_MILE_M,
                line.y_start_m,
                line.y_start_m / NAUTICAL_MILE_M,
                line.y_end_m,
                line.y_end_m / NAUTICAL_MILE_M,
                LINE_LENGTH_M,
                line.depth_m,
                line.west_half_width_m,
                line.east_half_width_m,
                line.coverage_width_m,
                line.west_edge_m,
                line.east_edge_m,
                line.spacing_from_previous_m,
                line.overlap_width_m,
                line.overlap_rate_pct,
            ]
        )

    for cell in detail[1]:
        cell.font = Font(bold=True)
        cell.fill = PatternFill("solid", fgColor="E2F0D9")
        cell.alignment = Alignment(horizontal="center", vertical="center")

```

```

for column in detail.columns:
    detail.column_dimensions[column[0].column_letter].width = 20
    for cell in column:
        cell.alignment = Alignment(horizontal="center", vertical="
center")
        if isinstance(cell.value, float):
            cell.number_format = "0.00"

checks = wb.create_sheet("相邻重叠校验")
checks.append(
    [
        "pair",
        "previous_line",
        "current_line",
        "spacing/m",
        "overlap_width/m",
        "current_W_h/m",
        "overlap_rate/%",
        "within_10_to_20_pct",
    ]
)
for line in lines[1:]:
    checks.append(
        [
            f"{line.index - 1}-{line.index}",
            line.index - 1,
            line.index,
            line.spacing_from_previous_m,
            line.overlap_width_m,
            line.coverage_width_m,
            line.overlap_rate_pct,
            MIN_OVERLAP_RATE * 100.0 <= line.overlap_rate_pct <=
MAX_OVERLAP_RATE * 100.0,
        ]
    )

```

```

for cell in checks[1]:
    cell.font = Font(bold=True)
    cell.fill = PatternFill("solid", fgColor="FCE4D6")
    cell.alignment = Alignment(horizontal="center", vertical="center")
for column in checks.columns:
    checks.column_dimensions[column[0].column_letter].width = 22
    for cell in column:
        cell.alignment = Alignment(horizontal="center", vertical="
center")
        if isinstance(cell.value, float):
            cell.number_format = "0.00"

wb.save(RESULT_XLSX)

def print_summary(lines: list[SurveyLine], summary: dict[str, float]) ->
None:
    print("Question 3 survey-line design")
    print(f"line_count: {len(lines)}")
    print(f"total_length_m: {summary['total_length_m']:.2f}")
    print(f"total_length_nm: {summary['total_length_nm']:.2f}")
    print(f"overlap_pct_range: {summary['min_overlap_pct']:.2f} - {summary
['max_overlap_pct']:.2f}")
    print(f"first_west_edge_m: {summary['first_west_edge_m']:.2f}")
    print(f"last_east_edge_m: {summary['last_east_edge_m']:.2f}")
    print(f"previous_count_gap_m: {summary['previous_count_gap_m']:.2f}")
    print("line x positions in meters:")
    print([round(line.x_m, 2) for line in lines])

def main() -> None:
    assert_flat_limit()
    lines = design_lines()
    summary = validate_design(lines)
    write_workbook(lines, summary)
    print_summary(lines, summary)

```

```
    print(f"Wrote {RESULT_XLSX}")

if __name__ == "__main__":
    main()
```